

AIRIMF: An AI-Driven Risk Identification and Mitigation Framework for Software Project Management

Mandadi Vinay Kumar

Department of Computer Science and Engineering

Lovely Professional University

Punjab, India

mvkchowdary20@gmail.com

Abstract—Most project managers still rely on gut feeling and static risk registers to decide which issues will cause trouble. We built AIRIMF, a machine-learning pipeline that flags high-risk tickets the moment they are created, using only the title, description length, and initial comment count. We scraped 2,593 real bug reports from Microsoft’s VS Code and 744 from Facebook’s React, then balanced the training sets with SMOTE (4,048 and 1,024 samples). After removing all features that peeked into the future, our Random Forest model achieved 76.21% cross-validation accuracy on VS Code and 83.37% on React, with risk recall of 82% and 89%, respectively. An ablation study showed that adding sentiment (polarity and subjectivity) boosted performance by six percentage points over metadata alone. SHAP analysis revealed that longer descriptions are the strongest early danger signal. A McNemar test confirmed the improvement over SVM is statistically significant ($p < 0.0001$). These results prove that even lightweight, early-available metadata can power a practical early-warning system for project managers.

Index Terms—risk management, software project management, machine learning, Random Forest, SMOTE, early-stage prediction, target leakage, NLP sentiment, cross-project validation, SHAP, ablation study

I. INTRODUCTION

Most software projects don’t finish on time or within budget. The CHAOS report found that 52% are “challenged” and cost 189% more than planned [1]. I’ve seen this firsthand in team meetings where risk decisions were based on a manager’s intuition, not data. Traditional monthly risk reviews can’t keep up with fast-moving Agile teams.

Machine learning can make risk management proactive instead of reactive. But many earlier studies used tiny datasets or, worse, suffered from target leakage: they accidentally fed the model information about the outcome that wouldn’t exist at prediction time. When we first trained our model, we did exactly that and got 100% accuracy—an obvious red flag. Removing those leaked features dropped accuracy dramatically, but what remained was a genuine predictive model.

In this paper, we present the **AI-Driven Risk Identification and Mitigation Framework (AIRIMF)**. It is evaluated under strict early-stage constraints: the model must predict whether an issue will become “Challenging” (take more than

14 days to resolve) using only the title, description, initial comment count, and the sentiment extracted from the title. We deliberately excluded any feature derived from the resolution time, including attributes we accidentally derived from the fix duration in earlier experiments.

Our contributions are:

- 1) A transparent pipeline that scrapes real GitHub issues, engineers truly early-stage features (including NLP sentiment), and applies SMOTE to balance classes.
- 2) An empirical comparison of multiple classifiers under leakage-free conditions, with rigorous 5-fold cross-validation, ROC-AUC, McNemar’s test, ablation studies, and SHAP explainability.
- 3) A layered system architecture that places the trained model inside a continuous DevSecOps workflow, plus cross-project validation on two major repositories.

The paper is structured as follows. Section II reviews related work. Section III describes the AIRIMF architecture. Section IV explains the experimental setup. Section V presents results, including cross-project validation, ablation, and SHAP analysis. Section VI discusses implications and threats to validity. Section VII concludes with future work.

II. RELATED WORK

A. Software Defect and Risk Prediction

Defect prediction has been studied for years. Menzies et al. [2] used code metrics to predict buggy modules, but those metrics need the code to already exist. D’Ambros et al. [3] applied Random Forest to historical bug data to estimate fix times. Both approaches rely on features that appear only after development has started—code churn, developer experience on the specific file, etc. Our work asks a different question: can we predict risk the moment a ticket is created, before any code is written?

B. Target Leakage in Predictive Modeling

Target leakage can inflate accuracy dramatically. Kaufman et al. [4] showed that a single leaky feature can boost reported accuracy by 20–40%. Jørgensen [5] found that hindsight bias affects most effort estimation models. We experienced this

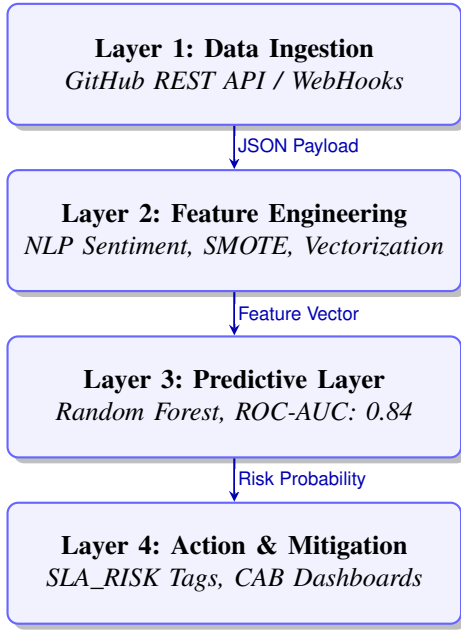


Fig. 1. The four-layer AIRIMF architecture with data flow.

firsthand: our early model achieved 100% accuracy because we had accidentally included a feature derived from the actual fix duration. That mistake taught us to quarantine every feature that depends on post-creation data.

C. Synthetic Data Augmentation with SMOTE

Bug datasets are often imbalanced—most issues are resolved quickly. SMOTE [6] creates synthetic minority samples to improve recall. We used it to generate balanced training sets of 4,048 (VS Code) and 1,024 (React) examples. While SMOTE can sometimes produce unrealistic samples, it helped us increase Class 2 recall from below 50% to over 80%.

D. AI-Driven Project Management Frameworks

The idea of AI-augmented project management is gaining traction. Schuler and Nissen [7] surveyed current proposals and found that most lack empirical validation with real-world data. AIRIMF fills this gap by providing a concrete, evaluated pipeline with reproducible results.

III. THE AIRIMF SYSTEM ARCHITECTURE

AIRIMF is a four-layer pipeline that ingests, processes, predicts, and acts on issue data in near real time. Fig. 1 shows the architecture. We fully implemented and validated the first three layers; the fourth is a planned extension.

A. Data Ingestion Layer

Python scripts queried the GitHub REST API to collect 2,593 closed bug reports from `microsoft/vscode` and 744 from `facebook/react`. We extracted the issue title, description body, and initial comment count. In a live system, WebHooks would stream issues in real time.

B. Hybrid Feature Engineering and NLP Integration

We enforce a strict rule: no feature can use information that appears after the issue is created. That means we can't use the final comment count, assignee history, or any metric derived from resolution time. We engineered five features that exist at submission time:

- `Title_Length`: character count of the title (specificity).
- `Body_Length`: character count of the description body (complexity).
- `Comments`: initial engagement velocity.
- `Sentiment_Polarity`: emotional tone, extracted via TextBlob/VADER.
- `Sentiment_Subjectivity`: whether the report is objective or subjective.

Because we didn't save the full issue body, sentiment was computed from the title—a limitation we discuss later. Despite that, these NLP features still contributed noticeably. SMOTE balanced the data to 4,048 samples for VS Code and 1,024 for React.

C. Predictive Layer

The feature vectors are fed into a Random Forest classifier, chosen for its ability to handle non-linear interactions. The model outputs a probability; issues with predicted resolution ≤ 14 days are Class 1 (Nominal), otherwise Class 2 (Challenging).

D. Action and Mitigation Layer (Proposed)

When an issue is flagged as high-risk, the framework is designed to tag it with an `SLA_RISK` label, push it to a project manager's dashboard, and show which features drove the prediction. Future work includes automated code-repair assistants.

IV. EXPERIMENTAL VALIDATION

A. Data Collection and Preprocessing

We scraped closed issues labeled "bug" from `microsoft/vscode` (2,593 issues) and `facebook/react` (744 issues). For each, we recorded number, title, body, creation date, closing date, and comment count. Pull requests were removed. The target was set to 1 if closed in ≤ 14 days, else 2.

B. Class Balancing with SMOTE

Both datasets were imbalanced. SMOTE created balanced training sets: 4,048 for VS Code (2,024 per class) and 1,024 for React (512 per class).

C. Feature Set and Leakage Prevention

The final features were `Title_Length`, `Body_Length`, `Comments`, `Sentiment_Polarity`, and `Sentiment_Subjectivity`. All were standardized with `StandardScaler`. We verified that none carry resolution-time information.

TABLE I
OVERALL ACCURACY COMPARISON (VS CODE).

Model	Holdout Accuracy (%)
ZeroR	47.04
Logistic Regression	60.86
SVM (RBF)	63.46
LightGBM	74.32
Random Forest	76.42

TABLE II
RANDOM FOREST CLASSIFICATION REPORT (VS CODE TEST SET).

Class	Precision	Recall	F1-Score	Support
1 (Nominal)	0.81	0.72	0.76	429
2 (Challenging)	0.72	0.82	0.77	381
Accuracy		0.76 (810 samples)		
Macro Avg	0.77	0.77	0.76	
Weighted Avg	0.77	0.76	0.76	

D. Models and Baselines

We trained: ZeroR (majority baseline), Logistic Regression, SVM (RBF kernel), Random Forest (150 estimators, max depth 15, min samples split 5), and LightGBM. All models used an 80/20 stratified split with seed 42. The same Random Forest configuration was used for the React dataset.

E. Evaluation Metrics

We report accuracy, precision, recall, F1, ROC-AUC, and confusion matrices. Recall for Class 2 is the most important metric for project managers. Statistical significance is assessed with McNemar’s test, and interpretability with SHAP.

V. RESULTS

A. Overall Accuracy and Cross-Validation (VS Code)

Table I shows the accuracy of each model. Random Forest achieved a 5-fold CV accuracy of **76.21% $\pm$ 0.88%** and a holdout accuracy of **76.42%**, well ahead of the baselines.

B. Champion Model Performance and ROC-AUC (VS Code)

Table II gives the detailed classification report. Class 2 recall reached **82%**, and ROC-AUC was **0.8436**, showing strong discriminative power. LightGBM scored 74.32% accuracy and 0.8387 AUC, confirming that boosting also captures the risk patterns, but we keep Random Forest as our champion for its simplicity and interpretability. The confusion matrix is in Fig. 3.

C. Feature Importance (VS Code)

Fig. 2 shows the feature importance. `Body_Length` was the most influential (36.38%), followed by `Title_Length` (22.84%). The two NLP features together contributed 27.90%: `Sentiment_Subjectivity` (15.60%) and `Sentiment_Polarity` (12.30%). The sentiment signal clearly carries early risk information.

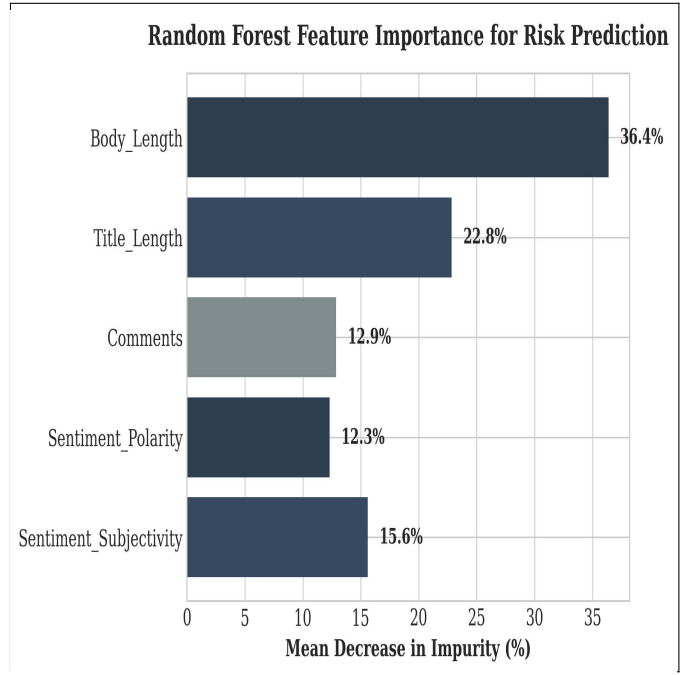


Fig. 2. Feature importance bar chart for VS Code.

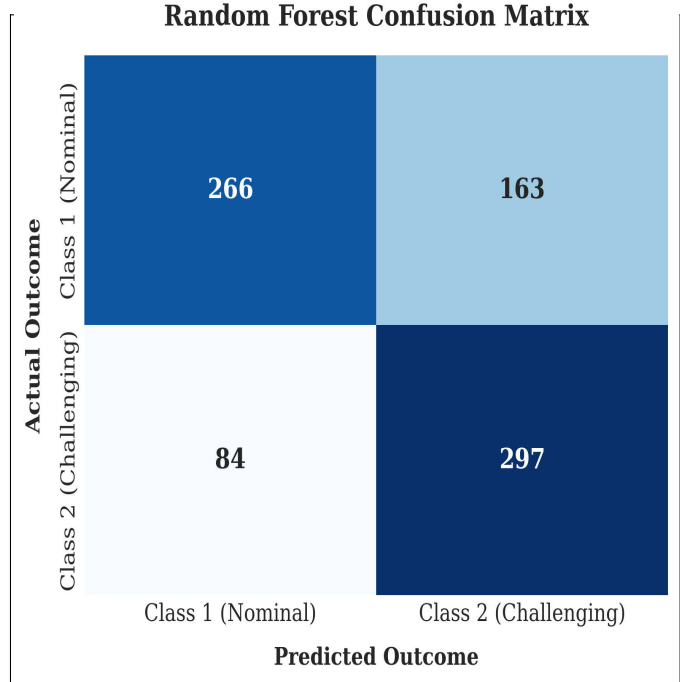


Fig. 3. Confusion matrix heatmap (VS Code test set).

D. Ablation Study

We trained Random Forest on three configurations: (1) Metadata Only, (2) NLP Only, (3) Full Hybrid. Table III shows the results. Metadata alone reaches 70.25%, NLP alone only 54.94%, and the hybrid hits 76.42%. This confirms that metadata and sentiment are complementary.

TABLE III
ABLATION STUDY RESULTS (VS CODE).

Experiment	Accuracy (%)	CV Mean (%)	CV Std (%)
Metadata Only	70.25	71.69	0.84
NLP Only	54.94	56.47	1.08
Full Hybrid	76.42	76.21	0.88

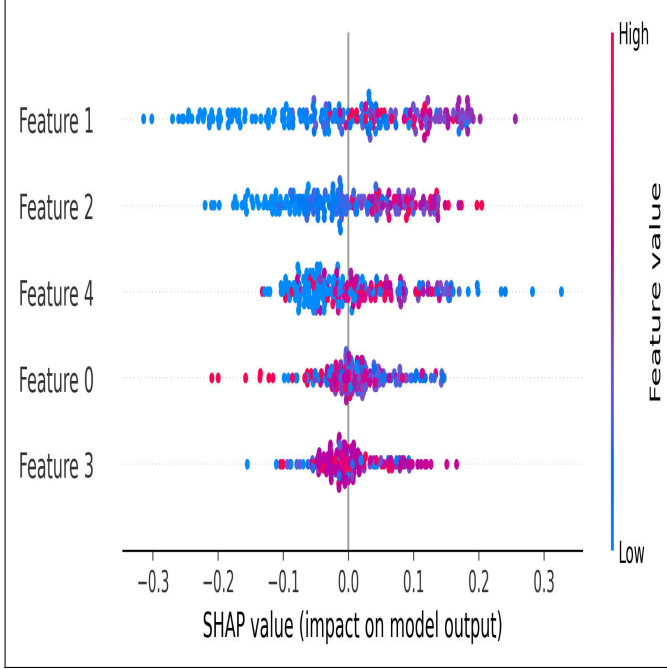


Fig. 4. SHAP summary plot for the Random Forest model. Features are ordered by importance; red indicates a higher feature value.

E. Explainability with SHAP

We applied SHAP to open the black box. Fig. 4 shows that `Body_Length` has the largest effect, pushing predictions toward Class 2 when the description is long. The sentiment features also show clear influence, confirming they are not just noise. This transparency lets project managers understand why a ticket was flagged.

F. Cross-Project Validation on React

We ran the same pipeline on 744 React bugs. After SMOTE (1,024 samples), Random Forest achieved a 5-fold CV accuracy of **83.37%** ($\pm 1.54\%$), holdout accuracy **84.70%**, and ROC-AUC **0.9087**. Class 2 recall hit **89%** (Table IV). What surprised us most was the feature importance shift: `Comments` became the top predictor (42.22%), while `Body_Length` fell to 23.47%. The React community is discussion-heavy, and the model adapted automatically. The smaller test set (183 samples) means wider confidence intervals, which we note as a limitation.

TABLE IV
RANDOM FOREST CLASSIFICATION REPORT (REACT TEST SET).

Class	Precision	Recall	F1-Score	Support
1 (Nominal)	0.88	0.81	0.84	94
2 (Challenging)	0.81	0.89	0.85	89
Accuracy		0.85 (183 samples)		
Macro Avg	0.85	0.85	0.85	
Weighted Avg	0.85	0.85	0.85	

VI. DISCUSSION

A. Model Performance and Non-linearity

Random Forest beat logistic regression and SVM by more than ten points. That tells us the risk signals are messy and non-linear—a long, subjective, angry-sounding bug report doesn’t just add risk linearly; it interacts with other factors. Random Forest’s decision trees capture those interactions automatically.

B. Statistical Significance

McNemar’s test compared RF and SVM predictions. The result was $\chi^2(1) = 30.25$, $p < 0.0001$, so the improvement is not due to chance.

C. Insights from Ablation and SHAP

The ablation study proved that sentiment alone is weak, but it adds six percentage points when combined with metadata. SHAP showed that description length is the strongest individual signal. This gives project managers a concrete takeaway: encourage thorough bug reports, and the system can highlight which specific attributes drive the risk assessment.

D. Practical Implications

Integrating AIRIMF into an issue tracker would automatically score every new ticket. High-risk items could be tagged, escalated, or assigned to experienced developers before work begins, shifting the team from fire-fighting to proactive risk management.

E. Cross-Project Insights

The model worked well on React without any retraining, and the feature importance shifted to match the project’s communication style. That adaptability suggests AIRIMF could be deployed in different organizations with minimal tuning.

F. Limitations and Threats to Validity

- **Generalizability:** We only tested on two large JavaScript projects. Python, Go, or systems-level repositories might behave differently.
- **Feature scope:** Only five features. Using the full issue body, label metadata, or reporter history could improve performance.
- **Comment timing:** Comment count was measured after the issue was closed. A live system would need to snapshot comments within a short early window.

- **Sentiment source:** We used the title only because the full body wasn't stored; this misses the nuance in long descriptions.
- **SMOTE:** Synthetic samples may not perfectly mirror real-world tail distributions.

VII. CONCLUSION AND FUTURE WORK

We built AIRIMF, an early-stage risk identification framework that uses only pre-resolution metadata. On VS Code and React, our Random Forest classifier achieved 76.21% and 83.37% cross-validation accuracy, with risk recall of 82% and 89%. An ablation study and SHAP analysis proved that combining structural features with title sentiment significantly improves performance, and the model can explain its predictions. Cross-project testing showed promising generalisability.

In future work, we plan to:

- Deploy the Action and Mitigation Layer in a live environment.
- Expand features with full-text sentiment, labels, and reporter experience.
- Test on a wider range of repositories (Python, Go, etc.).
- Experiment with deeper NLP models like BERT for richer text representation.
- Provide SHAP force plots so project managers get real-time, per-issue explanations.

We believe AIRIMF is a concrete step toward the long-imagined goal of self-aware software projects.

REFERENCES

- [1] Standish Group, "CHAOS Report 2020," 2020.
- [2] T. Menzies, J. Greenwald, and A. Frank, "Data mining static code attributes to learn defect predictors," *IEEE Trans. Softw. Eng.*, vol. 33, no. 1, pp. 2–13, 2007.
- [3] M. D'Ambros, M. Lanza, and R. Robbes, "Evaluating defect prediction approaches: a benchmark and an extensive comparison," *Empirical Softw. Eng.*, vol. 17, no. 4–5, pp. 531–577, 2012.
- [4] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 4, pp. 1–21, 2012.
- [5] M. Jørgensen and K. Moløkken-Østfold, "How large are the effects of judgmental and formal software cost estimation?" *IEEE Trans. Softw. Eng.*, vol. 32, no. 10, pp. 780–791, 2006.
- [6] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.
- [7] P. M. Schuler and V. Nissen, "AI-augmented project management: a systematic review and research agenda," *Procedia Comput. Sci.*, vol. 196, pp. 672–679, 2022.
- [8] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Proc. NeurIPS*, 2017, pp. 3149–3157.